Methods in Computational Sciences

Andreane Maure, UNI : am6823

2/23/2026

1 Preliminaries

Let A be invertible and let $\|\cdot\|$ be a consistent matrix norm, i.e.

$$\|Ax\| \leq \|A\|\|x\| \quad \text{for all } x.$$

The condition number of A is defined as

$$\kappa(A) = \|A\|\,\|A^{-1}\|.$$

(a)

Assume

$$Au = b, \qquad A\hat{u} = \hat{b}.$$

Subtracting the two equations gives

$$A(u - \hat{u}) = b - \hat{b}.$$

Since A is invertible,

$$u - \hat{u} = A^{-1}(b - \hat{b}).$$

Taking norms and using consistency,

$$\|u - \hat{u}\| \leq \|A^{-1}\|\,\|b - \hat{b}\|.$$

Now relate $\|u\|$ to $\|b\|$. From $b = Au$,

$$\|b\| = \|Au\| \leq \|A\|\|u\|,$$

which implies

$$\|u\| \geq \frac{\|b\|}{\|A\|}.$$

Therefore,

$$\frac{\|u - \hat{u}\|}{\|u\|} \leq \frac{\|A^{-1}\|\,\|b - \hat{b}\|}{\|b\|/\|A\|} = \|A\|\|A^{-1}\|\frac{\|b - \hat{b}\|}{\|b\|} = \kappa(A)\frac{\|b - \hat{b}\|}{\|b\|}.$$

$$\boxed{\frac{\|u - \hat{u}\|}{\|u\|} \leq \kappa(A)\frac{\|b - \hat{b}\|}{\|b\|}}$$

(b)

Define the forward error

$$e = u - \hat{u},$$

and the residual

$$r = b - A\hat{u}.$$

Since $b = Au$, we have

$$r = Au - A\hat{u} = A(u - \hat{u}) = Ae.$$

Upper bound : From $e = A^{-1}r$,

$$\|e\| \leq \|A^{-1}\|\,\|r\|.$$

Dividing by $\|u\|$ and using $\|u\| \geq \|b\|/\|A\|$,

$$\frac{\|e\|}{\|u\|} \leq \frac{\|A^{-1}\|\,\|r\|}{\|b\|/\|A\|} = \kappa(A)\frac{\|r\|}{\|b\|}.$$

Lower bound : From $r = Ae$,

$$\|r\| = \|Ae\| \leq \|A\|\|e\|,$$

which implies

$$\|e\| \geq \frac{\|r\|}{\|A\|}.$$

Since $u = A^{-1}b$,

$$\|u\| \leq \|A^{-1}\|\|b\|.$$

Therefore,

$$\frac{\|e\|}{\|u\|} \geq \frac{\|r\|/\|A\|}{\|A^{-1}\|\|b\|} = \frac{1}{\kappa(A)}\frac{\|r\|}{\|b\|}.$$

Combining both inequalities gives

$$\boxed{\frac{1}{\kappa(A)}\frac{\|r\|}{\|b\|} \leq \frac{\|e\|}{\|u\|} \leq \kappa(A)\frac{\|r\|}{\|b\|}}$$

The result shows that the relative error in the computed solution is controlled by the condition number of the matrix. More precisely, the forward error $\|e\|/\|u\|$ is proportional to the relative residual $\|r\|/\|b\|$, up to a factor $\kappa(A)$.
This means that if the matrix A is well-conditioned (that is, $\kappa(A)$ is close to 1), then a small residual guarantees that the computed solution is close to the exact solution. In that case, checking the residual is a reliable way to measure accuracy.
However, if A is ill-conditioned (large $\kappa(A)$), even a very small residual may correspond to a large error in the solution.

2 Eigenvalues and Condition Number of A

Let

$$A = \frac{1}{h^2}\,\text{tridiag}(-1, 2, -1)$$

be the $(m-1)\times(m-1)$ matrix arising from the centered finite difference discretization of $-u''(x)$ on $[0,1]$ with homogeneous Dirichlet boundary conditions. Here $h = \frac{1}{m}$ and $x_i = ih$ for $i = 1, \ldots, m-1$.

(a)

We claim that the eigenvectors of A are given by

$$v_i^{(j)} = \sin(j\pi x_i), \qquad j = 1, \ldots, m-1,$$

where $x_i = ih$.
Let $\theta = j\pi h = \frac{j\pi}{m}$. Then

$$v_i^{(j)} = \sin(i\theta).$$

Compute the i-th component of $Av^{(j)}$:

$$(Av^{(j)})_i = \frac{1}{h^2}\left(-v_{i-1}^{(j)} + 2v_i^{(j)} - v_{i+1}^{(j)}\right).$$

Using the trigonometric identity

$$\sin((i+1)\theta) + \sin((i-1)\theta) = 2\sin(i\theta)\cos\theta,$$

we obtain

$$-v_{i-1} + 2v_i - v_{i+1} = 2\sin(i\theta) - \big(\sin((i-1)\theta) + \sin((i+1)\theta)\big) = 2\sin(i\theta)(1-\cos\theta).$$

Therefore

$$(Av^{(j)})_i = \frac{2(1-\cos\theta)}{h^2}\sin(i\theta).$$

Thus

$$Av^{(j)} = \lambda_j v^{(j)},$$

so $v^{(j)}$ is an eigenvector.

(b)

From the previous computation,

$$\lambda_j = \frac{2(1-\cos\theta)}{h^2}, \qquad \theta = \frac{j\pi}{m}.$$

Using the identity

$$1-\cos\theta = 2\sin^2\left(\frac{\theta}{2}\right),$$

we obtain the standard form

$$\lambda_j = \frac{4}{h^2}\sin^2\left(\frac{j\pi}{2m}\right).$$

Since $h = \frac{1}{m}$,

$$\boxed{\lambda_j = 4m^2\sin^2\left(\frac{j\pi}{2m}\right), \qquad j = 1,\ldots,m-1.}$$

(c)

Because A is symmetric positive definite,

$$\kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{\lambda_{m-1}}{\lambda_1}.$$

Smallest eigenvalue : For $j = 1$,

$$\lambda_1 = 4m^2\sin^2\left(\frac{\pi}{2m}\right).$$

Using $\sin x \sim x$ as $x \to 0$,

$$\lambda_1 \sim 4m^2\left(\frac{\pi}{2m}\right)^2 = \pi^2.$$

Thus $\lambda_1 = \Theta(1)$.

Largest eigenvalue : For $j = m-1$,

$$\lambda_{m-1} = 4m^2\sin^2\left(\frac{(m-1)\pi}{2m}\right).$$

Since

$$\sin\left(\frac{(m-1)\pi}{2m}\right) = \cos\left(\frac{\pi}{2m}\right) \to 1,$$

we obtain

$$\lambda_{m-1} \sim 4m^2.$$

Thus $\lambda_{m-1} = \Theta(m^2)$.

Condition number : Therefore,

$$\kappa_2(A) = \frac{\lambda_{m-1}}{\lambda_1} \sim \frac{4m^2}{\pi^2}.$$

Hence,

$$\boxed{\kappa(A) = O(m^2) \quad \text{as } m \to \infty.}$$

3 Implementation of the BVP

(a)

Consider the boundary value problem

$$-u''(x) + \gamma u(x) = f(x), \qquad x \in [0,1],$$

with manufactured solution

$$u(x) = \sin(k\pi x) + c\left(x - \tfrac{1}{2}\right)^3,$$

where k is a positive integer and $c \in R$.

Computation of $f(x)$

First compute the derivatives of $u(x)$.

$$u'(x) = k\pi\cos(k\pi x) + 3c\left(x - \tfrac{1}{2}\right)^2,$$

$$u''(x) = -(k\pi)^2\sin(k\pi x) + 6c\left(x - \tfrac{1}{2}\right).$$

Substituting into the differential equation gives

$$f(x) = -u''(x) + \gamma u(x).$$

Therefore,

$$f(x) = -\left(-(k\pi)^2\sin(k\pi x) + 6c\left(x - \tfrac{1}{2}\right)\right) + \gamma\left(\sin(k\pi x) + c\left(x - \tfrac{1}{2}\right)^3\right).$$

Simplifying,

$$f(x) = \left((k\pi)^2 + \gamma\right)\sin(k\pi x) - 6c\left(x - \tfrac{1}{2}\right) + \gamma c\left(x - \tfrac{1}{2}\right)^3.$$

$$\boxed{f(x) = \left((k\pi)^2 + \gamma\right)\sin(k\pi x) - 6c\left(x - \tfrac{1}{2}\right) + \gamma c\left(x - \tfrac{1}{2}\right)^3}$$

(b) Modifications of `tri.c` to Solve the 1D BVP

Starting from the PETSc example `tri.c` (Chapter 2, *p4pdes*), I modified the code to solve the one-dimensional boundary value problem

$$-u''(x) + \gamma u(x) = f(x), \quad x \in (0,1), \quad u(0) = u(1) = 0,$$

using a second-order finite difference discretization on a uniform grid.

1. Grid Interpretation and Discretization

In `tri.c`, the parameter m represents the dimension of the linear system. In the modified code, m denotes the number of subintervals, and the number of grid points is

$$n = m + 1, \quad h = \frac{1}{m}.$$

The finite difference operator for interior points $i = 1, \ldots, m-1$ is assembled as

$$A_{i,i-1} = -\frac{1}{h^2}, \quad A_{i,i} = \frac{2}{h^2} + \gamma, \quad A_{i,i+1} = -\frac{1}{h^2},$$

which corresponds to the discrete operator $-u'' + \gamma u$.

2. Parallel Assembly

As in `tri.c`, the matrix and vectors are assembled in parallel. The ownership range is obtained via

`MatGetOwnershipRange(A,&Istart,&Iend)`,

and each MPI process inserts entries only for indices $i \in [\texttt{Istart}, \texttt{Iend}]$. The matrix and vectors are finalized with `MatAssemblyBegin/End` and `VecAssemblyBegin/End`.

3. Manufactured Exact Solution and Right-Hand Side

Unlike `tri.c`, where the right-hand side is computed as $b = Ax_{\text{exact}}$, I introduced a manufactured exact solution

$$u_{\text{exact}}(x) = \sin(k\pi x) + c(x - 0.5)^3,$$

and constructed the forcing term analytically from

$$f(x) = -u''_{\text{exact}}(x) + \gamma u_{\text{exact}}(x).$$

This allows direct verification of the numerical method by computing the relative error.

4. Boundary Conditions with Symmetry Preservation

Dirichlet boundary conditions are imposed using

`MatZeroRowsColumns(A, 2, bc, 1.0, uexact, f).`

This enforces $u(0) = u(1) = 0$ while preserving the symmetry of the matrix, as required in the assignment.

5. Relative Error Computation

After solving the linear system with `KSPSolve`, the relative error is computed as

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2}$$

and printed to the console.

(c)

We performed a convergence study for $\gamma = 0$ and $k = 1, 5, 10$ using mesh sizes

$$m = 40, 80, 160, 320, 640, 1280, \quad h = \frac{1}{m}.$$

For each mesh size, the relative L^2 error

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2}$$

was computed from the HDF5 output.
A log–log plot of the error versus h was generated. The slope was estimated by performing a linear regression on $(\log h, \log \text{error})$.
The observed orders of convergence are :

$$p \approx 2.00 \quad \text{for } k = 1,$$
$$p \approx 2.00 \quad \text{for } k = 5,$$
$$p \approx 2.01 \quad \text{for } k = 10.$$

This confirms that the centered finite difference discretization of $-u''$ is second-order accurate in the L^2 norm.

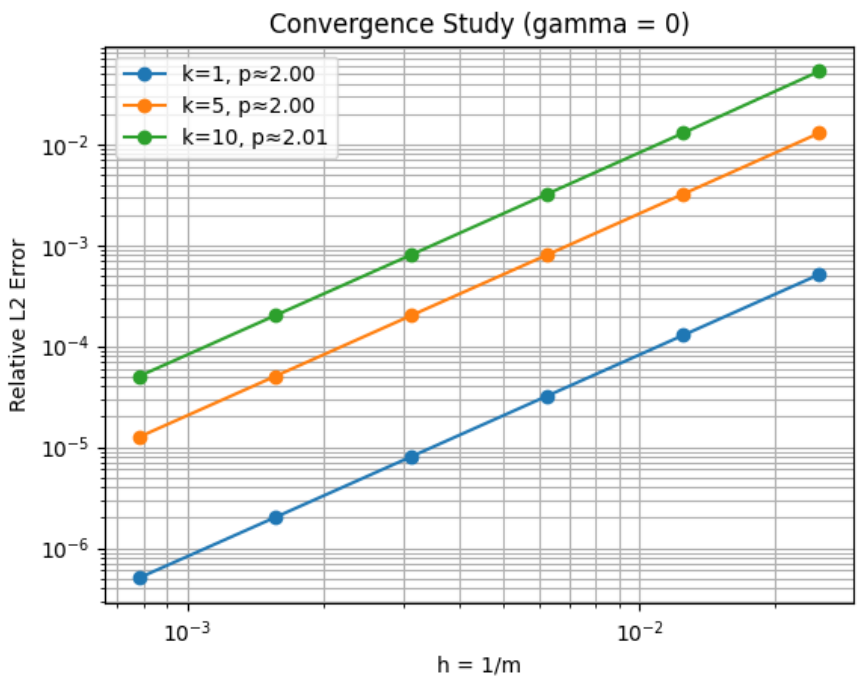

FIGURE 1 – Log–log plot of the relative L^2 error versus h for $\gamma = 0$ and $k = 1, 5, 10$. The slopes of the curves are approximately 2, confirming second-order convergence.

4

(a) Jacobi-preconditioned Richardson

We tested Jacobi-preconditioned Richardson using `-ksp_type richardson -pc_type jacobi`. With `-ksp_richardson_self` PETSc converged for a moderate grid size. For $m = 160$ ($h = 6.25 \times 10^{-3}$), the solver converged with `CONVERGED_RTOL` in 5703 iterations.

However, for the fine grid case ($m = 1280$, $h = 7.81 \times 10^{-4}$), the method did not reach the tolerance within the iteration limit (10000 iterations), terminating with `DIVERGED_ITS`.
This behavior is expected for Richardson-type stationary iterations. For Poisson-type operators ($\gamma = 0$), the condition number grows like $\kappa(A) = O(h^{-2})$ as $h \to 0$, so the spectral radius of the iteration matrix approaches 1 and convergence becomes extremely slow. As the mesh is refined, the iteration count increases rapidly and the method becomes impractical.

(b) Unpreconditioned Conjugate Gradient (1 process)

We solved the linear system with unpreconditioned Conjugate Gradient using a single MPI process (`-ksp_type cg -pc_type none`, `mpiexec -np 1`). For the default case ($m = 1280$), PETSc converged with `CONVERGED_RTOL` in 640 iterations.
CG is robust for SPD systems such as the discrete 1D Poisson operator. However, without preconditioning, the iteration count grows with mesh refinement because the condition number scales like $\kappa(A) = O(h^{-2})$ for $\gamma = 0$.

(c) Unpreconditioned Conjugate Gradient with $c = 0$

We repeated the unpreconditioned CG solve (`-ksp_type cg -pc_type none`) with $c = 0$. For $m = 1280$ and $k = 5$, PETSc converged with `CONVERGED_RTOL` in **1 iteration**, and the reported relative error was 1.25×10^{-5}.
This result can be explained spectrally. When $c = 0$, the manufactured exact solution is $u_{\text{exact}}(x) = \sin(k\pi x)$, which is an eigenfunction of the Dirichlet Laplacian. Consequently, its discrete counterpart is (approximately) an eigenvector of the finite-difference matrix A, i.e., $Au_{\text{exact}} \approx \lambda u_{\text{exact}}$. Since the right-hand side is constructed as $f = Au_{\text{exact}}$, the initial residual $r_0 = f - Au_0$ (with the default $u_0 = 0$) lies almost entirely in a single eigenmode. CG converges in at most the number of distinct eigenvalues present in the residual ; therefore, a single-mode residual yields convergence in one iteration.

(d) ICC-preconditioned Conjugate Gradient (1 process)

We solved the system with CG preconditioned by incomplete Cholesky (`-ksp_type cg -pc_type icc`) on one MPI process. For $m = 1280$, PETSc converged with `CONVERGED_RTOL` in **1 iteration** (and the reported relative error was $\approx 1.23 \times 10^{-5}$).
This very small iteration count indicates that the ICC preconditioner provides an extremely accurate approximation of the SPD matrix A. In the ideal case where the preconditioner equals the matrix ($M = A$), the preconditioned operator $M^{-1}A$ becomes the identity, and CG converges in a single step. For this 1D Poisson-type problem, A is tridiagonal and well structured, and incomplete Cholesky can be nearly exact, leading to a dramatically reduced effective condition number and convergence in one iteration. This contrasts with the unpreconditioned CG case, where the iteration count was much larger due to the $O(h^{-2})$ growth of $\kappa(A)$.

(e) Block Jacobi-preconditioned CG with 4 processes

We solved the system in parallel using 4 MPI processes with Conjugate Gradient preconditioned by Block Jacobi and an ICC sub-preconditioner on each block (`-ksp_type cg -pc_type bjacobi -sub_pc_type icc`). For $m = 1280$, PETSc converged with `CONVERGED_RTOL` in **6 iterations** (with relative error $\approx 6.58 \times 10^{-5}$).
Block Jacobi builds a block-diagonal preconditioner where each MPI rank applies a local preconditioner to its own subdomain. Using ICC as the sub-preconditioner provides a strong approximation on each block. However, unlike a global ICC factorization, the block-diagonal structure neglects the couplings across process boundaries. Therefore $M^{-1}A$ is not as close to the identity as in the single-process ICC case, and CG requires a small number of iterations (here 6) rather than converging in one step.

(f) MUMPS Direct Solver (1 and 4 Processors)

We solved the linear system using a direct solver by selecting `-ksp_type preonly -pc_type lu -pc_factor_solver_type mumps`. With `-ksp_type preonly`, no Krylov iterations are performed ; instead, the preconditioner is applied directly to solve the system. Since the preconditioner is LU factorization, this corresponds to a direct method.

1 MPI process : In the sequential case, the system is assembled and stored in `seqaij` format. The computed relative error is

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 1.230388 \times 10^{-5}.$$

4 MPI processes : In the parallel case, the system is assembled as a distributed matrix (`mpiaij` format). The computed relative error is the same :

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 1.230388 \times 10^{-5}.$$

Discussion : The identical relative errors confirm that the parallel run produces the same numerical solution as the sequential run. For this one-dimensional problem, the matrix is sparse and essentially tridiagonal, so the overall computational work is relatively small. Using multiple MPI processes introduces communication and synchronization overhead, which may limit parallel efficiency at this problem size. In general, direct solvers are robust and accurate, but their scalability depends on the problem structure and size ; for larger higher-dimensional problems, parallel factorization typically becomes more beneficial.